



Aras Innovator 33

File Handling

Document #: D-008108

Last Modified: 8/22/2024

Copyright Information

Copyright © 2024 Aras Corporation. All Rights Reserved.

Aras Corporation
100 Brickstone Square
Suite 100
Andover, MA 01810
Phone: 978-691-8900

E-mail: support@aras.com

Website: <https://www.aras.com/>

Notice of Rights

Copyright © 2024 by Aras Corporation and/or its affiliates. All rights reserved.

This document is protected by U.S. and international copyright laws and conventions. No copyright may be obscured or removed from this document. This document may not be modified or altered, or reproduced or transmitted in any form, without the explicit permission of the copyright holder.

Aras Innovator, Aras, and the Aras Corp "A" logo are registered trademarks of Aras Corporation in the United States and other countries.

All other trademarks referenced herein are the property of their respective owners.

Notice of Liability

THIS DOCUMENT IS PROVIDED FOR INFORMATIONAL PURPOSES ONLY, AND THE CONTENTS HEREOF ARE SUBJECT TO CHANGE WITHOUT NOTICE. THE INFORMATION CONTAINED IN THIS DOCUMENT IS DISTRIBUTED ON AN "AS IS" BASIS, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE OR A WARRANTY OF NON-INFRINGEMENT. ARAS SHALL HAVE NO LIABILITY TO ANY PERSON OR ENTITY WITH RESPECT TO ANY LOSS OR DAMAGE CAUSED OR ALLEGED TO BE CAUSED DIRECTLY OR INDIRECTLY BY THE INFORMATION CONTAINED IN THIS DOCUMENT OR BY THE SOFTWARE OR HARDWARE PRODUCTS DESCRIBED HEREIN.

Table of Contents

Send Us Your Comments	4
1 Overview.....	5
2 File Permissions	6
3 File Handling User Interface	7
3.1 Main Grid.....	7
3.2 Item Window.....	8
3.3 Relationship Grid.....	9
3.4 File History	9
3.4.1 <i>Enabling File History</i>	11
4 File API.....	12
4.1 Item.setFileProperty().....	12
4.2 Item.fetchFileProperty().....	12
4.3 replaceFile method.....	13
4.4 JavaScript functions	13
5 File Representations.....	15
5.1 AML API	15
5.2 Representation Characteristics.....	17
6 File UI Security.....	18

Send Us Your Comments

Aras Corporation welcomes your comments and suggestions on the quality and usefulness of this document. Your input is an important part of the information used for future revisions.

- Did you find any errors?
- Is the information clearly presented?
- Do you need more information? If so, where and what level of detail?
- Are the examples correct? Do you need more examples?
- What features did you like most?

If you find any errors or have any other suggestions for improvement, indicate the document title, and the chapter, section, and page number (if available).

You can send comments to us in the following ways:

Email:

TechDocs@aras.com

Subject: Aras Product Documentation

Or,

Postal service:

Aras Corporation
100 Brickstone Square
Suite 100
Andover, MA 01810
Attention: Aras Technical Documentation

If you would like a reply, provide your name, email address, address, and telephone number.

If you have usage issues with the software, visit <https://www.aras.com/support/>

1 Overview

The File Items in Aras Innovator are immutable. The following sections describe the File permission model, versioning design, UI, and History tracking.

The permission of a File Item is defined directly by the user's access to the File's container. The File container is the Item that has a Property of type Item with "File" as the data source. This includes the related_id Property of Relationship Items with File as the related ItemType.

Version control of File Items is also directly tied to the File container. The File ItemType itself is not versionable and the **update** action is not allowed for the File ItemType. In order to create a new version, the File's container should be versioned and the File Item replaced with a new File Item.

The File UI is consistent in all locations: the Form, the main grid, and the relationships grid. This UI gives options for viewing the File, downloading the File, and deleting the File from the Item's Property. It is important to note that deleting the File from the Item's Property will not automatically delete the File Item.

2 File Permissions

The permissions of a File Item are dependent on the Item containing that File, known as the File container. The File container is the Item which has a Property with a data source of the File ItemType. This includes relationship Items which have their `related_id` pointing to a File Item. This does not include Image properties used to display thumbnails on a Form. If a File has more than one container, the permissions will be based on the least restrictive access among all containers of the file.

There are two system File container objects that are used when a File is not referenced by any properties with a data source of the File ItemType. These are the “Orphaned Files Container” and the “Global Files Container”.

Files that are added with no container will automatically be placed in the Global Files Container, which allows **Get** for the World Identity. In other words, all Aras Innovator users will have access to these Files. In a standard implementation, this includes all Files stored in properties of type Image. To restrict access to these files, they can be added to another Item via a Property of type Item. Once added to another container, the File will be automatically removed from the Global Files Container and the containing Item will thereby define the permissions applied to the File.

Files that are originally added with a containing Item but are later orphaned in some way will automatically be placed in the Orphaned Files Container and will only be accessible to members of the File Administrators Identity. Members of the File Administrators Identity have unconditional access to all File Items. To restore access to these Files they can be added to another Item via a Property of type Item. Once added to another container, the File is automatically removed from the Orphaned Files Container and the containing Item will thereby define the permissions applied to the File.

In addition to the above permissions model, the following Variable Item may be defined to affect the global handling of files. This variable is not defined out of the box and a new Variable Item will need to be created if used. The Variable Item can be added in the Aras Innovator TOC Administration\Variables.

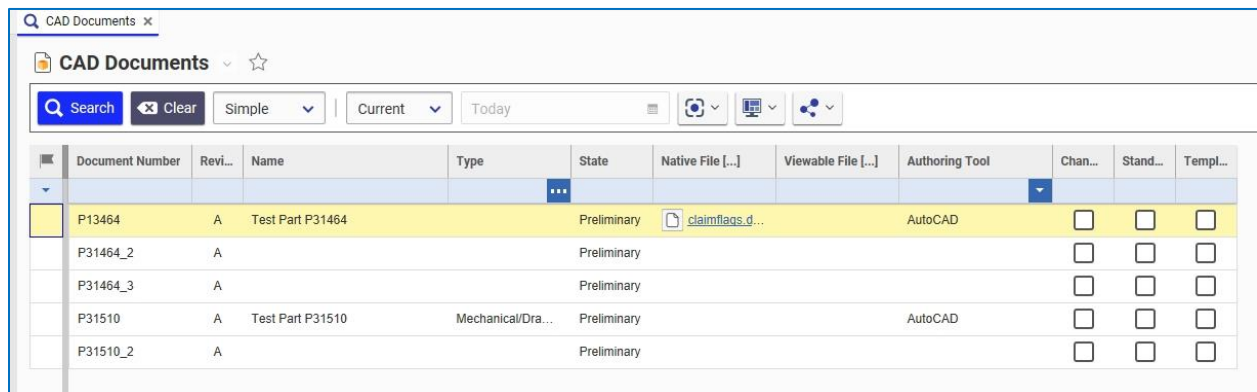
- `Force.Delete.Orphaned.Files`
 - If the value is set to 1, all files which get added to the Orphaned Files Container will automatically be deleted from Aras Innovator and the Vault.
 - If the value is set to 0 or if the Variable does not exist, all files which get added to the Orphaned Files Container will remain in the system.

3 File Handling User Interface

This section describes the File Handling UI in all the locations it is seen and used. This UI is designed to be consistent across all locations where Files can be seen.

3.1 Main Grid

The following screenshot shows the properties with a data source of the File ItemType that appear in the main grid.



Document Number	Revi...	Name	Type	State	Native File [...]	Viewable File [...]	Authoring Tool	Chan...	Stand...	Templ...
P13464	A	Test Part P13464		Preliminary	claimflags.d...		AutoCAD	☐	☐	☐
P13464_2	A			Preliminary				☐	☐	☐
P13464_3	A			Preliminary				☐	☐	☐
P31510	A	Test Part P31510	Mechanical/Dra...	Preliminary			AutoCAD	☐	☐	☐
P31510_2	A			Preliminary				☐	☐	☐

Figure 1.

Clicking on the name of the File will open the file within the browser and prompts you to “Open, Save, or Save As”. Clicking on the File icon to the left of the file name in the main grid opens the **Manage File** dialog.



Figure 2.

This dialog presents a similar File link as seen in the main grid as well as some additional information and buttons. You can choose to download the file to their downloads directory by clicking on the rightmost icon.

3.2 Item Window

The properties with a data source of the File ItemType that are displayed on the Item Form are seen as demonstrated by the Native File Property in Figure 3.

Figure 3.

This display is like the one seen in the main grid. Both the direct link to the File and the File icon are available. Clicking on the link streams the physical file to the browser to be handled while clicking on the File icon displays the Manage File UI.

If the Item is not claimed, you see the same dialog as when you clicked the file icon in the main grid. When the containing Item is claimed, additional buttons become available.



Figure 4.

Clicking on will clear the file from the Property. The File is not deleted unless the Variable Item `Force.Delete.Orphaned.Files` is set to 1 and the File is not included in any other Items.

Clicking on opens a file browser to select a new file for the Property. Any existing file stored in the property is replaced.

When you have finished working with the File, click on **OK** to close the dialog.

3.3 Relationship Grid

Files displayed in the Relationships grid work in the same way as those displayed in the main grid. By default, the `related_id` property is directly displayed in the relationships grid. This property acts as any other Property with a data source of the File ItemType.

Figure 5.

In the **Related File [...]** column, clicking on the file link streams the physical file to the browser to be handled while clicking on the File icon displays the Manage File UI. The available buttons in the UI are dependent on the current claim status of the Item as well as on the RelationshipType definition.

If the relationship does not allow the `related_id` to be null, then the **clear** icon is disabled.

If the relationship does not allow using **Pick Related**, then the **upload file** button is disabled.

Both buttons are disabled if the containing Item is claimed.

3.4 File History

File Items are handled in different ways in Aras Innovator. Because of this, the History Actions associated with Files are also different.

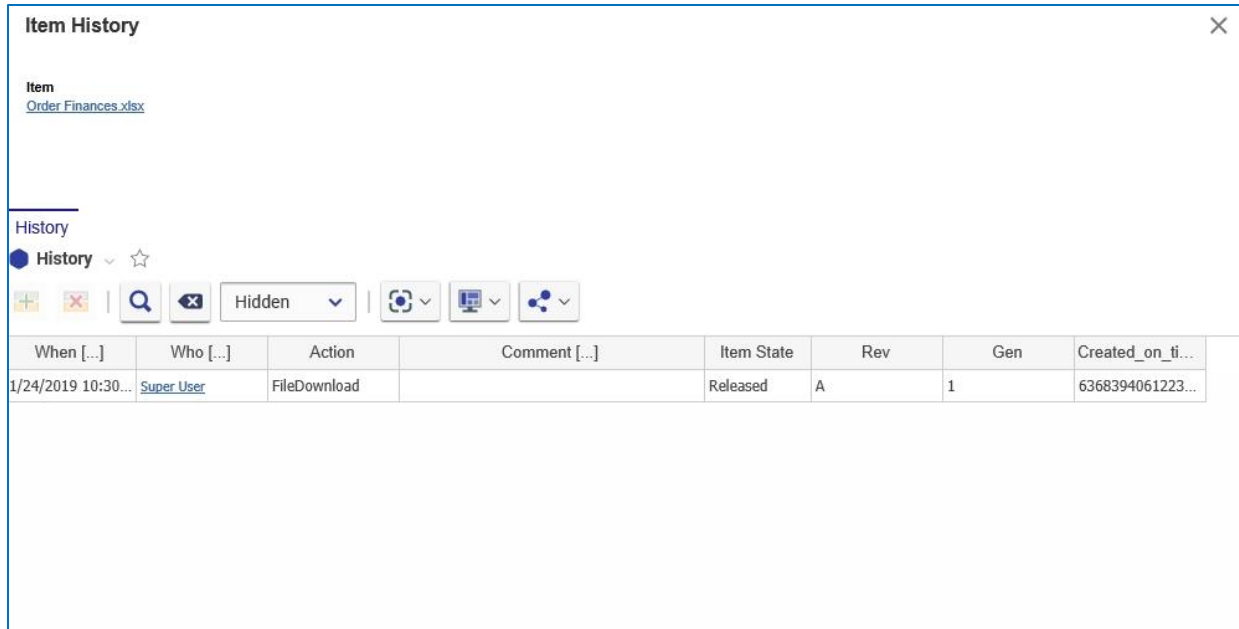


Figure 6.

The **FileDownload** History Action records a History entry whenever the File is requested from the server to be downloaded to the client. The standard **Add** History Action records a History entry whenever a new File Item is added to Aras Innovator.

Additional information on File Activity can be followed using the **Track** History option for the Properties themselves.

3.4.1 Enabling File History

In a standard installation of Aras Innovator, History for File Items is disabled.

To enable History, you need to login as Super User, login name **root**, and set the File History template on the File ItemType.

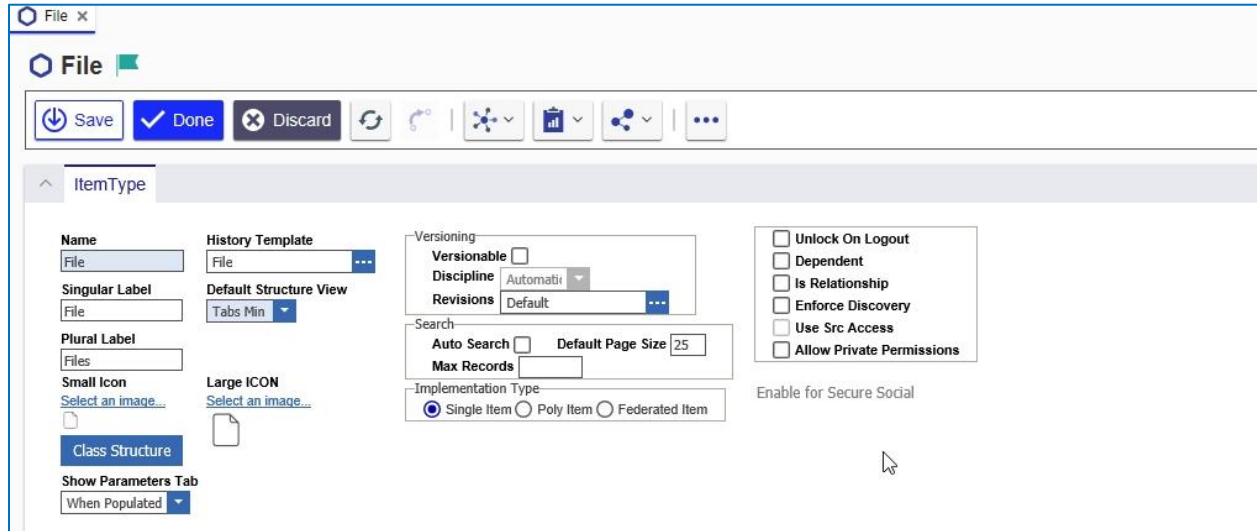


Figure 7.

Once enabled, access the File History from the Manage File dialog.



Figure 8.

4 File API

Note: It is recommended to make the Methods OS agnostic to use them with different operating systems. The main incompatibility issues in operating systems that need to be considered when implementing the method are related to file path case-sensitivity, file path separators, OS-specific line-endings, OS-specific code. For more information about cross-platform development please see section "2.3 Cross-platform development" in "Aras Innovator 33 - Programmer's Guide" document.

4.1 Item.setFileProperty()

The Item function `setFileProperty()` takes in 2 parameters: property and file path.

- **Property** – The property of type file for which you want to set the file item for add
- **File path** – The file path to the file on the file system

Example:

```
Item cadItem = inn.newItem("CAD", "add");  
cadItem.setFileProperty("native_file", filePath);  
...
```

4.2 Item.fetchFileProperty()

The function `fetchFileProperty()` takes in 3 parameters; property, file path, and mode. It returns *Item* of type *File*.

- **Property** – The property of type file for which you want to download the file
- **Target path** – The file path on the file system where to place the file
 - A file name can be specified as to change the name of the file i.e., C:\ArasTest\MyNewFileName.txt
- **Mode** – The mode of operation for checkout
 - `FetchFileMode.Normal` – In this mode, method performs file downloading to a defined target path. The return item has the `checkedout_path` property set to the value of the path where the file is downloaded.
 - `FetchFileMode.Dry` – In this mode, the method performs a “dry” run without downloading the physical file but checks if a file with same name exists in the specified target path. The return item has the `checkedout_path` property set to the value of the path to where the file would be downloaded. You can then use this path to determine if a file already exists in this path for further logic handling. No file is downloaded in this mode.

Example:

```
Item cadItem = inn.newItem("CAD", "add");

Item myFile = cadItem.fetchFileProperty("native_file", filePath,
FetchFileMode.Normal);

...
```

4.3 replaceFile method

A server method called `replaceFile` can be executed by File Administrators and replaces all references of the specified old file with the new file you specify.

This server method can be called with the following syntax:

```
<AML>

  <Item type="File" action="replaceFile" id="oldFileID" newId="newFileID" />

</AML>
```

4.4 JavaScript functions

To upload a File using method code on the client-side, the following examples may be referenced.

To download a File through javascript, use the `downloadFile` function. For example:

```
aras.downloadFile(myFileItem.node);
```

Client-side Action to prompt for File:

```
function processAddingFile(file) {
    var newFile = aras.newItem("File", file);

    if (newFile) {
        aras.itemsCache.addItem(newFile);
        var res = aras.saveItemEx(newFile);
        aras.unlockItemEx(res);
    }
}

aras.vault.selectFile().then(processAddingFile.bind(this));
```

Form onClick event for Button:

```
function processAddingFile(file) {
    var newFile = aras.newItem("File", file);

    if (newFile) {
        aras.itemsCache.addItem(newFile);
        window.handleItemChange("myFileProperty", newFile);
    }
}

aras.vault.selectFile().then(processAddingFile.bind(this));
```

Client-side setFileProperty Example:

```
aras.vault.selectFile().then(function (fileObject)
{
    var d = aras.IomInnovator.newItem('CAD', 'add');
    d.setProperty('item number', 'CAD_001');
    d.setFileProperty('native_file', fileObject);
    d.applyAsync();
});
```

5 File Representations

Starting in Aras Innovator 11.0 SP11, it is possible to represent files with the contents of another file, which is available to the end user. The most common use cases for this feature are:

- Providing a web-capable image for another, less available file type. For example, a CGM file cannot be displayed by browsers, but could be represented by an SVG file.
- Providing multiple resolutions of an image. For example, a high-resolution JPEG could be used as the main image on an item. This image then could have multiple representations at different resolutions, which could be used as a thumbnail or a mobile view.

5.1 AML API

It is possible to use standard AML to operate on File Representations.

AML to Add a File Representation to a File.

```
<AML>

  <Item type="fr_Representation" action="add">
    <source_id>{source file ID}</source_id>
    <representation_type_id>
      <Item type="fr_RepresentationType" action="get">
        <name>XML</name>
      </Item>
    </representation_type_id>
    <Relationships>
      <Item type="fr_RepresentationFile" action="add">
        <kind>js</kind>
        <related_id>{ID of representation JS file}</related_id>
      </Item>
      <Item type="fr_RepresentationFile" action="add">
        <kind>html</kind>
        <related_id>{ID of representation HTML file}</related_id>
      </Item>
    </Relationships>
  </Item>
</AML>
```

AML to Get a File Representation to a File.

```
<AML>

  <Item type="fr_Representation" action="get" select="id">
    <source_id>{source file ID}</source_id>
    <representation_type_id>
      <Item type="fr_RepresentationType" action="get">
        <name>XML</name>
      </Item>
    </representation_type_id>
    <Relationships>
      <Item type="fr_RepresentationFile" select="related_id(id,
filename)">
      </Item>
    </Relationships>
  </Item>
</AML>
```

AML to Delete a File Representation to a File.

The clearest way to delete representations is to first get the representations, and then delete them in a separate action.

```
<AML>

  <Item type="fr_Representation" action="get" select="id">
    <source_id>[source file ID]</source_id>
    <representation_type_id>
      <Item type="fr_RepresentationType" action="get">
        <name>XML</name>
      </Item>
    </representation_type_id>
  </Item>
</AML>

<AML>

  <Item type="fr_Representation" action="delete" idlist="%idlist%"></Item>
</AML>
```


5.2 Representation Characteristics

File Representations make use of the new XProperty model to distinguish between different representations of the same RepresentationType. For example, a resolution characteristic may be defined to allow different values for different representations within a RepresentationType.

Note that there is a known issue that modifications to the RepresentationType can take a significant amount of server processing (see the Release Notes for details on IR-053121). This will cause the server to appear frozen to end users.

We recommend performing any modifications to characteristics as an offline action. Additionally, the database should be backed up prior to modifying the characteristics.

6 File UI Security

The Aras Innovator user interface has a security mechanism to prevent loading of files directly in the browser and will force the file to download to local disk first instead.

The Vault Server vault.config file contains the Content Security Policy (CSP) that prevents the loading and executing of files in the user interface.

The policy has 2 possible settings:

- 1) **attachment** – This will force a file to be downloaded in the UI

```
<mimeType value="application/zip">
  <headers>
    <add name="Content-Disposition" value="attachment" />
  </headers>
</mimeType>
```

- 2) **inline** – This will force a file to load in the browser but will limit the execution of scripts in the file when loading in the browser.

```
<mimeType value="text/xml">
  <headers>
    <add name="Content-Disposition" value="inline" />
    <add name="Content-Security-Policy" value="sandbox" />
  </headers>
</mimeType>
```

When loading as “inline” the Content-Security-Policy header enables a sandbox for the requested resource. This restricts page actions including the prevention of pop-ups and the execution of plugins and scripts. More about Content-Security-Policy is here: <https://content-security-policy.com/>.